

Part 15: Revision Sheet

Everything on a few pages, for the night before

Terminology in one box

root	topmost node, no parent
leaf	BOTH children are NULL
internal	not a leaf
ancestor	any node above you on the path to the root
subtree	a node plus ALL of its descendants
level	distance from the root, root is level 0
depth	edges from the root down to the node (measured downward from root)
height	edges from the node down to its deepest leaf (measured upward from leaves)

N nodes => N - 1 edges

perfect tree of height h => $2^{(h+1)} - 1$ nodes, level d holds 2^d nodes

balanced => height about $\log_2(N)$

skewed => height N - 1

BST: for EVERY node, whole left subtree < node < whole right subtree

=> inorder of a BST is sorted

Height counting convention: with $h(\text{NULL}) = 0$ you are counting nodes, with $h(\text{NULL}) = -1$ you are counting edges. LeetCode counts nodes.

Node definition

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
```

The master template

```

ReturnType solve(TreeNode* root) {
    if (root == NULL) return answerForEmptyTree;
    ReturnType L = solve(root->left);
    ReturnType R = solve(root->right);
    return combine(L, R, root->val);
}

```

Base case identities:

```

sum -> 0          count -> 0          max -> INT_MIN      min -> INT_MAX
all -> true       any -> false       height(nodes) -> 0  height(edges) -> -1

```

Recursive traversals

```

void pre(TreeNode* r, vector<int>& v) {
    if (!r) return;
    v.push_back(r->val); pre(r->left, v); pre(r->right, v);
}
void in(TreeNode* r, vector<int>& v) {
    if (!r) return;
    in(r->left, v); v.push_back(r->val); in(r->right, v);
}
void post(TreeNode* r, vector<int>& v) {
    if (!r) return;
    post(r->left, v); post(r->right, v); v.push_back(r->val);
}

```

On the standard tree:

```

      1
     /\
    2 3
   /\  \
  4 5  6
pre  : 1 2 4 5 3 6
in   : 4 2 5 1 3 6
post : 4 5 2 6 3 1
bfs  : 1 2 3 4 5 6

```

Iterative traversals

```

// PREORDER: pop, record, push right then left
stack<TreeNode*> st; st.push(root);
while (!st.empty()) {
    TreeNode* n = st.top(); st.pop();
    out.push_back(n->val);
    if (n->right) st.push(n->right);
    if (n->left)  st.push(n->left);
}

// INORDER: dive left, pop, record, go right
stack<TreeNode*> st; TreeNode* cur = root;
while (cur || !st.empty()) {

```

```

while (cur) { st.push(cur); cur = cur->left; }
cur = st.top(); st.pop();
out.push_back(cur->val);
cur = cur->right;
}

// POSTORDER: preorder with left/right swapped, then reverse
stack<TreeNode*> st; st.push(root);
while (!st.empty()) {
    TreeNode* n = st.top(); st.pop();
    out.push_back(n->val);
    if (n->left) st.push(n->left);
    if (n->right) st.push(n->right);
}
reverse(out.begin(), out.end());

```

BFS template

```

vector<vector<int>>> levelOrder(TreeNode* root) {
    vector<vector<int>>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);

    while (!q.empty()) {
        int sz = q.size(); // snapshot, never q.size() in the condition
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop();
            level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        res.push_back(level);
    }
    return res;
}

```

Variations, all inside the inner loop:

```

right view : if (i == sz - 1) res.push_back(n->val);
left view  : if (i == 0)      res.push_back(n->val);
level sum   : sum += n->val;
level max   : mx = max(mx, n->val);
zigzag      : build level normally, reverse it on alternate rows
bottom-up   : build normally, reverse(res) at the end

```

Height template

```

int height(TreeNode* r) { // in nodes

```

```

    if (!r) return 0;
    return 1 + max(height(r->left), height(r->right));
}

```

Bottom-up template

```

int dfs(TreeNode* r) {
    if (!r) return 0;
    int L = dfs(r->left);
    int R = dfs(r->right);
    best = max(best, /* answer THROUGH r, uses L and R */);
    return /* value for the PARENT, uses only one of L, R */;
}

```

Diameter template

```

int best = 0;
int height(TreeNode* r) {
    if (!r) return 0;
    int L = height(r->left), R = height(r->right);
    best = max(best, L + R);          // edges of the path turning here
    return 1 + max(L, R);
}
// answer = best

```

Balanced template

```

int check(TreeNode* r) {                // height, or -1 if unbalanced
    if (!r) return 0;
    int L = check(r->left); if (L == -1) return -1;
    int R = check(r->right); if (R == -1) return -1;
    if (abs(L - R) > 1) return -1;
    return 1 + max(L, R);
}
// balanced <=> check(root) != -1

```

Path templates

```

// root to leaf, target
bool hasPathSum(TreeNode* r, int t) {
    if (!r) return false;
    t -= r->val;
    if (!r->left && !r->right) return t == 0;
    return hasPathSum(r->left, t) || hasPathSum(r->right, t);
}

// all root to leaf paths, with backtracking

```

```

void dfs(TreeNode* r, vector<int>& path, vector<vector<int>>& res) {
    if (!r) return;
    path.push_back(r->val);
    if (!r->left && !r->right) res.push_back(path);
    else { dfs(r->left, path, res); dfs(r->right, path, res); }
    path.pop_back(); // undo
}

// maximum path sum, any node to any node
int gain(TreeNode* r) {
    if (!r) return 0;
    int L = max(0, gain(r->left));
    int R = max(0, gain(r->right));
    best = max(best, r->val + L + R); // record
    return r->val + max(L, R); // return
}
// best starts at INT_MIN

```

View templates

```

// TOP / BOTTOM view: BFS carrying horizontal distance
map<int,int> col;
queue<pair<TreeNode*,int>> q; q.push({root, 0});
while (!q.empty()) {
    auto c = q.front(); q.pop();
    TreeNode* n = c.first; int hd = c.second;

    if (col.find(hd) == col.end()) col[hd] = n->val; // TOP: first wins
    // col[hd] = n->val; // BOTTOM: last wins

    if (n->left) q.push({n->left, hd - 1});
    if (n->right) q.push({n->right, hd + 1});
}
// read col in key order

// VERTICAL order: map<int, vector<int>> col; col[hd].push_back(n->val);

// RIGHT view by DFS: visit right first, record when level == res.size()

```

```

// BOUNDARY traversal
bool isLeaf(TreeNode* n) { return !n->left && !n->right; }

// 1. root, if it is not a leaf
// 2. left boundary top down, skipping leaves
//     cur = root->left; go left if possible, else right
// 3. all leaves, left to right (simple DFS)
// 4. right boundary bottom up, skipping leaves
//     cur = root->right; go right if possible, else left; collect then reverse

```

Horizontal distance rule: $hd(\text{left}) = hd - 1$, $hd(\text{right}) = hd + 1$, $hd(\text{root}) = 0$.

Morris template

```

TreeNode* cur = root;
while (cur) {
    if (!cur->left) {
        out.push_back(cur->val);           // preorder records here too
        cur = cur->right;
    } else {
        TreeNode* pred = cur->left;
        while (pred->right && pred->right != cur) pred = pred->right;

        if (!pred->right) {
            // out.push_back(cur->val);    // uncomment for PREORDER
            pred->right = cur;             // create thread
            cur = cur->left;
        } else {
            pred->right = NULL;            // remove thread
            out.push_back(cur->val);        // INORDER records here
            cur = cur->right;
        }
    }
}

```

$O(N)$ time, $O(1)$ extra space, temporarily modifies the tree.

Complexity cheat sheet

Algorithm	Time	Space	Note
Recursive traversal	$O(N)$	$O(H)$	call stack
Iterative traversal	$O(N)$	$O(H)$	explicit stack
Morris traversal	$O(N)$	$O(1)$	modifies tree temporarily
BFS level order	$O(N)$	$O(W)$	W is max width, up to $N/2$
Height, count, sum	$O(N)$	$O(H)$	one pass
Diameter, naive	$O(N^2)$	$O(H)$	height recomputed
Diameter, bottom-up	$O(N)$	$O(H)$	
Balanced, naive	$O(N^2)$	$O(H)$	
Balanced, sentinel	$O(N)$	$O(H)$	
All root to leaf paths	$O(N \cdot H)$	$O(H)$	copying each path
Max path sum	$O(N)$	$O(H)$	
Views with a map	$O(N \log N)$	$O(N)$	$O(N)$ with a vector and offset
Boundary traversal	$O(N)$	$O(H)$	
Build from preorder + inorder	$O(N)$	$O(N)$	needs the index map
Serialize / deserialize	$O(N)$	$O(N)$	
Subtree of another tree	$O(N \cdot M)$	$O(H)$	
BST search, insert, LCA	$O(H)$	$O(1)$ iterative	$O(\log N)$ if balanced

H is the height: $O(\log N)$ balanced, $O(N)$ skewed.

Which approach should I think of first

1. Read the statement. Circle the words: level, view, path, subtree, ancestor, column, nearest, sorted, BST.
2. levels / rows / views by level / nearest
 top / bottom / vertical / column
 path that may turn
 root to leaf / needs context from above
 needs facts from both subtrees
 sorted / kth / validate / search
 "without recursion"
 "O(1) extra space" and inorder
 - > BFS, size snapshot
 - > BFS + horizontal distance + map
 - > bottom-up, record vs return
 - > preorder + extra parameter
 - > postorder, combine(L, R, node)
 - > BST, use inorder or the ordering
 - > explicit stack
 - > Morris
3. Write the base case first. What is the answer for an empty tree?
4. Say in one sentence what the function returns.
5. Ask: am I calling a subtree-walking helper inside a recursion that already walks the subtree? If yes, merge the two passes.
6. State the complexity. Compare with the constraints. Then code.

The mistakes that cost the most marks

missing NULL base case
 leaf test written with || instead of &&
 "missing child" treated as a leaf (minimum depth)
 target checked at NULL instead of at a leaf (path sum)
 missing & on the output vector
 q.size() used in the loop condition instead of a snapshot
 returning L + R upward in diameter or max path sum
 best initialised to 0 instead of INT_MIN in max path sum
 recomputing height at every node -> $O(N^2)$
 forgetting pop_back in backtracking
 global not reset between test cases
 unordered_map used where the output must be ordered by column
 right boundary not reversed
 root printed twice in boundary traversal
 Morris thread not removed
 assuming a plain binary tree is a BST

The four questions to ask on any new tree problem

1. What information does each subtree need to return?

2. Is this DFS, BFS, or bottom-up?
3. What should my base case be?
4. What do I combine from the left and the right?

If you can answer these four for a problem you have never seen, you have learned binary trees.